

Claude-powered paper audit pipeline — design report

Repo: `sednabcn/LLM-HypatiX-DEV` **Scope:** audits `*/*_patched.tex` against `hypatiax/data/results/`, using the Claude API, independent of the existing `ci_paper_audit.yml` gate.

1. Why this exists

`ci_paper_audit.yml` already does deterministic, rule-based auditing of the paper against results data. This pipeline adds a second, LLM-driven layer on top, for two jobs a deterministic script can't do:

Numeric consolidation — when a paper claim's underlying results have multiple files (reruns, seeds, shards), decide the single authoritative value and *why*, then propose (and in narrow cases, auto-commit) the fix.

The narrative "open/flagging points" report — a human-readable account of what's stale, what's missing, and what's actively blocked, on top of the raw findings JSON.

It deliberately does **not** try to replicate the deep, cascading, multi-file investigations you've been running manually (the `68_74-issue.txt` / `leak_report.tex` style sessions). Those stay human+Claude, outside CI.

This pipeline's only relationship to that work is: **it reads the block list those sessions produce, and it never overwrites a number in a table/figure that list says is currently invalid.**

2. Two-tier design

	Tier 1 — automated, in CI	Tier 2 — human+Claude, outside CI
What it is	<code>llm_audit_inspector.py</code> , one Claude call per claim, bounded	The kind of session in <code>68_74-issue.txt</code> / <code>notes-nguyen12.txt</code> — iterative, requests more files, hypotheses get overturned, scope legitimately expands
Trigger	push to <code>*_patched.tex</code> / <code>hypatiax/data/results/**</code> / <code>audit/observations/**</code> , or manual dispatch	you, manually, whenever
Output	<code>logs/llm_audit_findings.json</code> , updates to <code>audit_registry.json</code> , three forms of report via	Incident docs like <code>leak_report.tex</code> , filed back in <code>audit/observations/*.json</code> (<code>type: "block"</code>)
Can it auto-commit?	Yes, but only a single-digit numeric substitution, mechanically re-verified (no LLM trust)	Never automatically — always a human-reviewed patch
Relationship between them	Reads Tier 2's block list before doing anything; refuses to touch any claim inside a blocked table/figure	Produces the block list; the only way to remove a block is editing <code>audit_registry.json</code> directly

3. Data flow



push to *_patched.tex / hypatiax/data/results/** / audit/observations/**



1. INSPECT	llm_audit_inspector.py
- extract_claims()	reads: *_patched.tex,
- merge audit_registry	hypatiax/data/results/**,
(seed/tier2 preserved)	audit/observations/*,
- blocklist check	audit_registry.json (existing)
- consolidate() per	calls: consolidate_results.py (Claude API)
unblocked claim	writes: logs/llm_audit_findings.json,
	audit_registry.json (merged)



2. PATCH	llm_patch_apply.py
generate: findings →	reads: logs/llm_audit_findings.json
patch_manifest.yaml	writes: patch_manifest.yaml (proposed
entries	entries, all kinds)
apply-numeric: verify +	reads/writes: patch_manifest.yaml,
apply ONLY	*_patched.tex (numeric_* only)
numeric_consolidated/	auto-commits ONLY these, mechanically
numeric_from_upload	re-verified line-by-line



3. REPORT	generate_llm_audit_report.py
writes all 3 forms	reads: logs/llm_audit_findings.json,
	audit_registry.json



audit/llm_ logs/llm_audit_ <dir>/<file>_llm_audit.md
audit_report report_<run_id>. (per *_patched.tex,
.md (cumula- md (artifact committed next to it)
tive, only, not
committed) committed)

Separately, whenever you've run a Tier-2 investigation and produced a block:



you file audit/observations/<name>.json (type: "block")



next Tier-1 run's INSPECT step merges it into audit_registry.json
as a tier2_upload entry → its `blocks` list is now enforced

4. File inventory — what goes where in the repo

All paths below are relative to the repo root.

#	File	Repo destination	Purpose
1	<code>ci_llm_paper_audit.yml</code>	<code>.github/workflows/ci_llm_paper_audit.yml</code>	The workflow itself: 3 jobs (<code>inspect</code> → <code>patch</code> → <code>report</code>), independent of <code>ci_paper_audit.yml</code> Tier-1 entry point. Extracts numeric claims from <code>*_patched.tex</code>, tracks table/figure labels, checks the blocklist, calls <code>consolidate_results.py</code> for unblocked claims, merges results into <code>audit_registry.json</code> The only file that calls the Claude API. One bounded call per claim: given a section's result summary files, returns the authoritative value + method + confidence + provenance as strict JSON Deterministic, no Claude calls. <code>generate</code> subcommand writes findings into <code>patch_manifest.yaml</code> (your existing schema); <code>apply-numeric</code> subcommand mechanically verifies and applies <i>only</i> numeric single-token substitutions Writes the open/flagging-points report in all three requested forms (cumulative, timestamped, per-tex-file)
2	<code>llm_audit_inspector.py</code>	<code>scripts/patches/llm_audit_inspector.py</code>	
3	<code>consolidate_results.py</code>	<code>scripts/patches/consolidate_results.py</code>	
4	<code>llm_patch_apply.py</code>	<code>scripts/patches/llm_patch_apply.py</code>	
5	<code>generate_llm_audit_report.py</code>	<code>scripts/patches/generate_llm_audit_report.py</code>	

#	File	Repo destination	Purpose
6	<code>audit_registry_lib.py</code>	<code>scripts/patches/audit_registry_lib.py</code>	Shared library: load/save <code>audit_registry.json</code> , compute the blocklist, merge Tier-1 findings without ever touching seed/ <code>tier2_upload</code> entries
7	<code>seed_audit_registry.py</code>	<code>scripts/patches/seed_audit_registry.py</code>	One-time (idempotent) importer — merges a seed file into <code>audit_registry.json</code>
8	<code>plan_action_seed.json</code>	<code>audit/seed/plan_action_seed.json</code>	The actual 13 <code>PLAN-ACTION.txt</code> items + item 14 (the ground-truth leak from <code>leak_report.tex</code>), pre-converted to the registry schema. Run <code>seed_audit_registry.py</code> once against this to populate the registry with your real, current findings
9	<code>README.md</code> (observations)	<code>audit/observations/README.md</code>	Documents both upload paths (committed <code>.md</code> / <code>.json</code> , and <code>workflow_dispatch</code> ad-hoc) and the <code>type: "block"</code> schema for filing Tier-2 findings

Files this pipeline *generates* at runtime (not delivered by me — created by the workflow)

File	Path	Committed?
<code>audit_registry.json</code>	repo root	Yes — merged and committed each run
<code>patch_manifest.yaml</code>	repo root	Yes — your existing file, this pipeline only appends to it
<code>logs/llm_audit_findings.json</code>	<code>logs/</code>	No — artifact only
<code>audit/llm_audit_report.md</code>	<code>audit/</code>	Yes — cumulative report, overwritten each run
<code>logs/llm_audit_report_<run_id>.md</code>	<code>logs/</code>	No — artifact only
<code><dir>/<texfile>_llm_audit.md</code>	next to each <code>*_patched.tex</code>	Yes
<code>audit/incident_reports/*.tex</code>	<code>audit/incident_reports/</code>	You commit these manually when a Tier-2 session produces one (e.g. <code>leak_report.tex</code>) — this pipeline only reads/links to them, never writes them

5. Setup steps (in order)

Copy files 1–9 above into the listed paths.

`pip install anthropic pyyaml` (already in the workflow's install step — nothing to do if you're only running it in CI).

Confirm `secrets.ANTHROPIC_API_KEY` is set at the repo level (it already is, per `ci_paper_audit.yml`).

Seed the registry once:




```
bash
python3 scripts/patches/seed_audit_registry.py \
  --seed audit/seed/plan_action_seed.json \
  --registry audit_registry.json
git add audit_registry.json && git commit -m "chore: seed audit_registry.json from PLAN-ACTION.txt"
```

This immediately puts Tables 9–13, 15, `tab:hybrid_all`, `tab:runtime`, `tab:timing`, and Figures 9–13 on the blocklist, and items #4, #10, #12, #13 in as `resolved` so Tier 1 doesn't re-flag them. 5. If you have

`audit/incident_reports/leak_report.tex` already, commit it now too — the seed entry for item 14 already points `linked_report` at that exact path. 6. **Fix the one assumption I made blind:**

`resolve_section_to_results()` in `llm_audit_inspector.py` assumes a `paper_targets.json` schema I invented (`{"sections": [{"section": ..., "experiment_ids": [...]}]}`). Point me at the real file (or the equivalent

lookup already inside `hypatia_inspector.py`) and I'll rewire that one function — everything else is independent of it.

6. Known limitations (found via testing, not theoretical)

Label detection is regex-based, not a real tex parser. `LABEL_RE` catches `\label{tab:...}`/`\label{fig:...}`; `PROSE_LABEL_RE` catches literal `"Table N"` / `"Figure N"`. It already correctly caught `1.73x` inside a `Table 9` context in testing and blocked it — but if your papers reference tables/figures some other way (e.g. `\cref`, `Tab.~\ref{...}`), extend those two patterns.

Numeric extraction was buggy on first pass and has been fixed and re-tested: the original regex missed `"1.73x"` (a trailing letter broke the word-boundary check) — it now explicitly handles `%`, `x/x`, and `s/sec` suffixes. Re-verify against a few of your real `_patched.tex` files before trusting it broadly, since paper prose has more unit variety than the smoke test covered.

False positives are cheap, false negatives are not. The pipeline is deliberately conservative — e.g. it also flagged the bare `9` in "Table 9" as a separate (blocked) claim, which is harmless noise, not a wrong auto-fix.

7. Still open

Real `paper_targets.json` schema (item 6 above) — blocks Tier 1's section→results resolution from working correctly until fixed.

Whether you want `audit_registry.json`'s `resolved`/`open` items (2, 3, 6, 7, 8, 9, 11) to also get Tier-1 numeric checks run against them now, or left for manual/Tier-2 handling since several are pure text fixes with no numeric component.